In-Class Board Problems

Student answer sheets | Algorithms & Data Structures

Use one page per problem. Solve it at the board in chalk first. Then copy your Python solution here, and fill in the analysis.

1. Phones and computers are locked away. Work individually.
2. Write a plain Python function. Use only what Python gives you, plus collections and heapq.
3. State the time complexity and the auxiliary space in Big-O. Auxiliary space is the extra memory beyond the input; what you build and return counts.
4. Estimate the memory for the input size on the page: the input, your auxiliary data, and the total, in bytes. State your assumptions.
5. Stuck? Ask for help after at least 2 failed attempts, and write them in the box on the page before you ask.

Memory reference (CPython 3.13, 64-bit): int 28 bytes; float 24; str 41 + 1 per character; list 56 + 8 per slot; tuple 40 + 8 per item; dict about 50 per entry; set about 42 per element; deque about 8.3 per element; node object about 100; recursive call about 150. Small ints (-5 to 256), True, False and None are shared.

Name _____ Date _____ Class 1, Problem A

Class 1: Big-O and Python memory
Problem A (easier)

#1480 Running Sum of 1d Array

Easy. https://leetcode.com/problems/running-sum-of-1d-array/

Given a list of integers nums, return a list where entry i is the sum of nums[0] through nums[i] (the running sum).

```
Example: nums = [1, 2, 3, 4]  ->  [1, 3, 6, 10]
Start with:  def runningSum(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 distinct ints above 256 (so none are cached small ints).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 1, Problem B

Class 1: Big-O and Python memory
Problem B (harder)

#217 Contains Duplicate

Easy. https://leetcode.com/problems/contains-duplicate/

Given a list of integers nums, return True if any value appears at least twice, and False if every value is distinct.

```
Example: nums = [1, 2, 3, 1]  ->  True        nums = [1, 2, 3, 4]  ->  False
Start with:  def containsDuplicate(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 distinct ints above 256.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 2, Problem A

Class 2: Arrays and in-place updates
Problem A (easier)

#283 Move Zeroes

Easy. https://leetcode.com/problems/move-zeroes/

Given a list nums, move all 0s to the end while keeping the order of the non-zero values. You must change the list in place, without making a copy.

```
Example: nums = [0, 1, 0, 3, 12]  ->  [1, 3, 12, 0, 0]
Start with:  def moveZeroes(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 ints, half of them 0.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 2, Problem B

Class 2: Arrays and in-place updates
Problem B (harder)

#88 Merge Sorted Array

Easy. https://leetcode.com/problems/merge-sorted-array/

nums1 has m real values in sorted order followed by n zeros of spare room (length m + n). nums2 has n values in sorted order. Merge nums2 into nums1 so that nums1 is sorted. Change nums1 in place.

```
Example: nums1 = [1, 2, 3, 0, 0, 0], m = 3, nums2 = [2, 5, 6], n = 3  ->  nums1 = [1, 2, 2, 3, 5, 6]
```

```
Start with:  def merge(nums1, m, nums2, n):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for m = n = 50,000, so nums1 has 100,000 slots and nums2 has 50,000.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 3, Problem A

Class 3: Strings and prefix sums
Problem A (easier)

#14 Longest Common Prefix

Easy. https://leetcode.com/problems/longest-common-prefix/

Given a list of strings strs, return the longest string that is a prefix of every string. If there is none, return the empty string "".

```
Example: strs = ["flower", "flow", "flight"]  ->  "fl"        strs = ["dog", "racecar", "car"]  ->  ""
```

```
Start with:  def longestCommonPrefix(strs):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for m = 200 strings of 200 characters each, all identical in the worst case (constraints allow up to 200 strings of up to 200 characters).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 3, Problem B

Class 3: Strings and prefix sums
Problem B (harder)

#303 Range Sum Query - Immutable

Easy. https://leetcode.com/problems/range-sum-query-immutable/

Design a class NumArray. It is built once from a list nums. Then sumRange(left, right) is called many times and must return the sum of nums[left] through nums[right], inclusive.

```
Example: NumArray([-2, 0, 3, -5, 2, -1]);  sumRange(0, 2) -> 1;  sumRange(2, 5) -> -1;  sumRange(0, 5) -> -3
```

```
Start with:  class NumArray
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 values (the LeetCode limit), large enough that every prefix sum is a new 28-byte int above 256.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 4, Problem A

Class 4: Hash sets and hash maps
Problem A (easier)

#242 Valid Anagram

Easy. https://leetcode.com/problems/valid-anagram/

Given two strings s and t of lowercase letters, return True if t is an anagram of s (it uses exactly the same letters with the same counts), and False otherwise.

```
Example: s = "anagram", t = "nagaram"  ->  True        s = "rat", t = "car"  ->  False
```

```
Start with:  def isAnagram(s, t):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 50,000 characters in each string (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 4, Problem B

Class 4: Hash sets and hash maps
Problem B (harder)

#1 Two Sum

Easy. https://leetcode.com/problems/two-sum/

Given a list of integers nums and an integer target, return the indexes of the two numbers that add up to target. Exactly one such pair exists, and you may not use the same position twice.

```
Example: nums = [2, 7, 11, 15], target = 9  ->  [0, 1]
Start with:  def twoSum(nums, target):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 distinct ints above 256 (the LeetCode limit), with the answer at the very end so the dict grows to n entries.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 5, Problem A

Class 5: Two pointers
Problem A (easier)

#125 Valid Palindrome

Easy. https://leetcode.com/problems/valid-palindrome/

A phrase is a palindrome if, after making every letter lowercase and removing every character that is not a letter or digit, it reads the same forward and backward. Given a string s, return True if it is a palindrome.

```
Example: s = "A man, a plan, a canal: Panama"  ->  True        s = "race a car"  ->  False
Start with:  def isPalindrome(s):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 200,000 characters (the LeetCode limit), all letters, in a palindrome (the worst case, checked all the way to the middle).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 5, Problem B

Class 5: Two pointers
Problem B (harder)

#977 Squares of a Sorted Array

Easy. https://leetcode.com/problems/squares-of-a-sorted-array/

Given a list nums sorted in non-decreasing order (it may contain negative numbers), return a new list of the squares of every number, also sorted in non-decreasing order.

```
Example: nums = [-4, -1, 0, 3, 10]  ->  [0, 1, 9, 16, 100]
Start with:  def sortedSquares(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 values (the LeetCode limit), values in the thousands so every square is a new 28-byte int above 256.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 6, Problem A

Class 6: Sliding window
Problem A (easier)

#643 Maximum Average Subarray I

Easy. https://leetcode.com/problems/maximum-average-subarray-i/

Given a list nums of integers and an integer k, find the contiguous subarray of length k with the largest average, and return that average.

```
Example: nums = [1, 12, -5, -6, 50, 3], k = 4  ->  12.75   (the window [12, -5, -6, 50])
Start with:  def findMaxAverage(nums, k):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 ints above 256 (the LeetCode limit is 100,000 values); the running sum is a new int each step but only one is alive at a time.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 6, Problem B

Class 6: Sliding window
Problem B (harder)

#121 Best Time to Buy and Sell Stock

Easy. https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

prices[i] is the price of a stock on day i. Choose one day to buy and a later day to sell. Return the largest possible profit, or 0 if no profit is possible.

```
Example: prices = [7, 1, 5, 3, 6, 4]  ->  5   (buy at 1, sell at 6)
prices = [7, 6, 4, 3, 1]  ->  0
```

```
Start with:  def maxProfit(prices):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 prices (the LeetCode limit) above 256.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 7: Stacks
Problem A (easier)

#20 Valid Parentheses

Easy. https://leetcode.com/problems/valid-parentheses/

Given a string s containing only the characters ( ) [ ] { }, return True if it is valid: every opening bracket is closed by the same type of bracket, and brackets close in the correct order.

```
Example: s = "()[]{}"  ->  True        s = "(]"  ->  False        s = "([)]"  ->  False
```

```
Start with:  def isValid(s):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 characters (the LeetCode limit), all "(" (the worst case for the stack).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 7, Problem B

Class 7: Stacks
Problem B (harder)

#844 Backspace String Compare

Easy. https://leetcode.com/problems/backspace-string-compare/

Two strings s and t contain lowercase letters and the character #, which means backspace (it deletes the previous character, and does nothing if there is none). Return True if the two strings are equal after all backspaces are applied.

```
Example: s = "ab#c", t = "ad#c"  ->  True   (both become "ac")        s = "a#c", t = "b"  ->  False
```

```
Start with:  def backspaceCompare(s, t):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = m = 200 characters (the LeetCode limit), no # characters (the worst case for stack size).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 8, Problem A

Class 8: Queues and deques
Problem A (easier)

#933 Number of Recent Calls

Easy. https://leetcode.com/problems/number-of-recent-calls/

Design a class RecentCounter. ping(t) records a request at time t (in milliseconds; each t is larger than the one before) and returns how many requests were made in the last 3000 ms, that is, in the range [t - 3000, t], including this one.

```
Example: ping(1) -> 1;  ping(100) -> 2;  ping(3001) -> 3;  ping(3002) -> 3
Start with:  class RecentCounter
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for 10,000 pings, one per millisecond (the worst case for window size: 3001 entries).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 8, Problem B

Class 8: Queues and deques
Problem B (harder)

#232 Implement Queue using Stacks

Easy. https://leetcode.com/problems/implement-queue-using-stacks/

Implement a first-in, first-out queue using only two stacks (Python lists used with append and pop at the end). Provide push(x), pop(), peek() and empty().

```
Example: push(1); push(2); peek() -> 1; pop() -> 1; empty() -> False
Start with:  class MyQueue
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 pushes (far more than the LeetCode limit of 100 calls, to show growth).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 9, Problem A

Class 9: Monotonic stack and deque
Problem A (easier)

#496 Next Greater Element I

Easy. https://leetcode.com/problems/next-greater-element-i/

nums1 is a subset of nums2; all values are distinct. For each x in nums1, find x in nums2 and return the first value to the right of it in nums2 that is greater than x, or -1 if there is none. Return the answers in the order of nums1.

```
Example: nums1 = [4, 1, 2], nums2 = [1, 3, 4, 2]  ->  [-1, 3, -1]
Start with:  def nextGreaterElement(nums1, nums2):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = m = 1,000 (the LeetCode limit), nums2 a random permutation, nums1 equal to nums2.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 9, Problem B

Class 9: Monotonic stack and deque
Problem B (harder)

#739 Daily Temperatures

Medium. https://leetcode.com/problems/daily-temperatures/

temperatures[i] is the temperature on day i. Return a list answer where answer[i] is the number of days you must wait after day i to get a warmer temperature. If there is no future warmer day, answer[i] = 0.

```
Example: temperatures = [73, 74, 75, 71, 69, 72, 76, 73]  ->  [1, 1, 4, 2, 1, 1, 0, 0]
```

```
Start with:  def dailyTemperatures(temperatures):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 days (the LeetCode limit), all the same temperature (the worst case for stack size: nobody is ever popped).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 10, Problem A

Class 10: Linked lists
Problem A (easier)

#206 Reverse Linked List

Easy. https://leetcode.com/problems/reverse-linked-list/

Given the head of a singly linked list, reverse the list and return the new head.

```
Example: 1 -> 2 -> 3 -> 4 -> 5  becomes  5 -> 4 -> 3 -> 2 -> 1
Start with:  def reverseList(head):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 5,000 nodes (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 10, Problem B

Class 10: Linked lists
Problem B (harder)

#19 Remove Nth Node From End of List

Medium. https://leetcode.com/problems/remove-nth-node-from-end-of-list/

Given the head of a singly linked list and an integer n, remove the nth node from the end of the list and return the head. (n = 1 means the last node.) n is always valid.

```
Example: head = 1 -> 2 -> 3 -> 4 -> 5, n = 2  ->  1 -> 2 -> 3 -> 5
Start with:  def removeNthFromEnd(head, n):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for L = 30 nodes (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 11, Problem A

Class 11: Recursion
Problem A (easier)

#509 Fibonacci Number

Easy. https://leetcode.com/problems/fibonacci-number/

The Fibonacci numbers start F(0) = 0, F(1) = 1, and every later number is the sum of the two before it: F(n) = F(n - 1) + F(n - 2). Given n (0 <= n <= 30), return F(n).

```
Example: n = 2  ->  1        n = 3  ->  2        n = 4  ->  3
Start with:  def fib(n):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 30 (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 11, Problem B

Class 11: Recursion
Problem B (harder)

#50 Pow(x, n)

Medium. https://leetcode.com/problems/powx-n/

Implement pow(x, n): compute x raised to the integer power n, where x is a float and n can be negative or zero. (n is between -2^31 and 2^31 - 1, so a loop that multiplies x by itself n times is far too slow.)

```
Example: x = 2.0, n = 10  ->  1024.0        x = 2.0, n = -2  ->  0.25
Start with:  def myPow(x, n):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 2^31, so the recursion is about 32 frames deep, each about 150 bytes, plus a few floats of 24 bytes each.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 12: Bit manipulation
Problem A (easier)

#136 Single Number

Easy. https://leetcode.com/problems/single-number/

In a list nums every value appears exactly twice except one value that appears once. Return the value that appears once, using only O(1) extra space.

```
Example: nums = [4, 1, 2, 1, 2]  ->  4
Start with:  def singleNumber(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 30,000 values (the LeetCode limit), distinct values above 256 appearing in pairs.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 12, Problem B

Class 12: Bit manipulation
Problem B (harder)

#260 Single Number III

Medium. https://leetcode.com/problems/single-number-iii/

In a list nums exactly two values appear once and every other value appears exactly twice. Return the two values that appear once (in any order). Use O(n) time and O(1) extra space.

```
Example: nums = [1, 2, 1, 3, 2, 5]  ->  [3, 5]
Start with:  def singleNumber(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 30,000 values (the LeetCode limit), distinct values in pairs plus two singletons.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 13, Problem A

Class 13: Sorting
Problem A (easier)

#1331 Rank Transform of an Array

Easy. https://leetcode.com/problems/rank-transform-of-an-array/

Given a list arr, replace each element by its rank: the smallest value has rank 1, the next larger distinct value has rank 2, and so on. Equal values share the same rank. Return the list of ranks.

```
Example: arr = [40, 10, 20, 30]  ->  [4, 1, 2, 3]        arr = [37, 12, 28, 9, 100, 56, 80, 5, 12]  ->  [5, 3, 4, 2, 8, 6, 7, 1, 3]
```

```
Start with:  def arrayRankTransform(arr):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 distinct ints above 256.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 13, Problem B

Class 13: Sorting
Problem B (harder)

#912 Sort an Array

Medium. https://leetcode.com/problems/sort-an-array/

Given a list nums, sort it in ascending order and return it. You must not use the built-in sort. The solution should run in O(n log n) time.

```
Example: nums = [5, 2, 3, 1]  ->  [1, 2, 3, 5]        nums = [5, 1, 1, 2, 0, 0]
->  [0, 0, 1, 1, 2, 5]
Start with:  def sortArray(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 50,000 ints (the LeetCode limit), random order.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 14: Binary search
Problem A (easier)

#704 Binary Search

Easy. https://leetcode.com/problems/binary-search/

Given a list nums sorted in ascending order with distinct values, and an integer target, return the index of target, or -1 if it is not in the list. The solution must run in O(log n) time.

```
Example: nums = [-1, 0, 3, 5, 9, 12], target = 9  ->  4        target = 2  ->  -1
Start with:  def search(nums, target):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 sorted distinct ints above 256 (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 14: Binary search
Problem B (harder)

#34 Find First and Last Position of Element in Sorted Array

Medium. https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/

Given a list nums sorted in non-decreasing order (duplicates allowed) and a target, return [first, last], the first and last index at which target appears, or [-1, -1] if it does not appear. Run in O(log n) time.

```
Example: nums = [5, 7, 7, 8, 8, 10], target = 8  ->  [3, 4]        target = 6 ->  [-1, -1]
Start with:  def searchRange(nums, target):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 sorted ints above 256 (the LeetCode limit), with a block of 1,000 repeated target values.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 15, Problem A

Class 15: Trees: traversal
Problem A (easier)

#104 Maximum Depth of Binary Tree

Easy. https://leetcode.com/problems/maximum-depth-of-binary-tree/

Given the root of a binary tree, return its maximum depth: the number of nodes along the longest path from the root down to a leaf.

```
Example: root = [3, 9, 20, null, null, 15, 7]  ->  3        root = []  ->  0
Start with:  def maxDepth(root):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 nodes in a balanced tree (height 14).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 15, Problem B

Class 15: Trees: traversal
Problem B (harder)

#102 Binary Tree Level Order Traversal

Medium. https://leetcode.com/problems/binary-tree-level-order-traversal/

Given the root of a binary tree, return its level order traversal: a list of lists, where list i holds the values of all nodes at depth i, from left to right.

```
Example: root = [3, 9, 20, null, null, 15, 7]  ->  [[3], [9, 20], [15, 7]]
Start with:  def levelOrder(root):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 nodes in a complete tree (14 levels, the last level about 1,700 to 5,000 nodes).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 16: Binary search trees
Problem A (easier)

#700 Search in a Binary Search Tree

Easy. https://leetcode.com/problems/search-in-a-binary-search-tree/

Given the root of a binary search tree and a value val, return the node whose value equals val, or None if there is no such node. (The result is the subtree rooted at that node.)

```
Example: root = [4, 2, 7, 1, 3], val = 2  ->  the subtree [2, 1, 3]        val = 5  ->  None
Start with:  def searchBST(root, val):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 5,000 nodes (the LeetCode limit) in a balanced BST.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 16, Problem B

Class 16: Binary search trees
Problem B (harder)

#98 Validate Binary Search Tree

Medium. https://leetcode.com/problems/validate-binary-search-tree/

Given the root of a binary tree, return True if it is a valid binary search tree: for every node, all values in its left subtree are strictly smaller and all values in its right subtree are strictly larger (and both subtrees are valid).

```
Example: root = [2, 1, 3]  ->  True        root = [5, 1, 4, null, null, 3, 6]
->  False   (3 is in the right subtree of 5 but is smaller than 5)
```

```
Start with:  def isValidBST(root):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 nodes in a balanced valid BST (height 14).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 17: Heaps and priority queues
Problem A (easier)

#703 Kth Largest Element in a Stream

Easy. https://leetcode.com/problems/kth-largest-element-in-a-stream/

Design a class KthLargest. It is created with an integer k and an initial list nums. add(val) adds val to the stream and returns the kth largest element of everything seen so far (the kth largest in sorted order, counting duplicates).

```
Example: KthLargest(3, [4, 5, 8, 2]);  add(3) -> 4;  add(5) -> 5;  add(10) -> 5;  add(9) -> 8;  add(4) -> 8
```

```
Start with:  class KthLargest
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for k = 10,000 (the LeetCode limit) with a stream of 100,000 distinct ints above 256 created by the caller.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 17, Problem B

Class 17: Heaps and priority queues
Problem B (harder)

#347 Top K Frequent Elements

Medium. https://leetcode.com/problems/top-k-frequent-elements/

Given a list nums and an integer k, return the k most frequent values (in any order). The answer is guaranteed to be unique. Better than O(n log n) is expected.

```
Example: nums = [1, 1, 1, 2, 2, 3], k = 2  ->  [1, 2]
Start with:  def topKFrequent(nums, k):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 values, all distinct (d = n), and k = 100.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 18, Problem A

Class 18: Graphs and depth-first search
Problem A (easier)

#733 Flood Fill

Easy. https://leetcode.com/problems/flood-fill/

image is a grid of integers (a picture of pixel colors). Starting from the pixel (sr, sc), change its color and the color of every pixel connected to it through up, down, left or right moves, as long as those pixels have the same starting color, to a new color. Return the changed image.

```
Example: image = [[1, 1, 1], [1, 1, 0], [1, 0, 1]], sr = 1, sc = 1, color = 2
->  [[2, 2, 2], [2, 2, 0], [2, 0, 1]]
Start with:  def floodFill(image, sr, sc, color):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for A 50 x 50 grid (the LeetCode limit, n = 2,500 cells), all one color (the worst case).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 18: Graphs and depth-first search
Problem B (harder)

#200 Number of Islands

Medium. https://leetcode.com/problems/number-of-islands/

Given a grid of the characters "1" (land) and "0" (water), count the islands. An island is a group of land cells connected horizontally or vertically. The edges of the grid are surrounded by water.

```
Example: grid = [["1","1","0","0","0"], ["1","1","0","0","0"],
["0","0","1","0","0"], ["0","0","0","1","1"]]  ->  3
```

```
Start with:  def numIslands(grid):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for A 300 x 300 grid (the LeetCode limit, n = 90,000 cells) that is all land, so one island.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 19, Problem A

Class 19: Breadth-first search
Problem A (easier)

#1971 Find if Path Exists in Graph

Easy. https://leetcode.com/problems/find-if-path-exists-in-graph/

There are n vertices numbered 0 to n - 1 and a list of undirected edges. Return True if there is a path from source to destination, and False otherwise.

```
Example: n = 3, edges = [[0, 1], [1, 2], [2, 0]], source = 0, destination = 2 ->  True        n = 6, edges = [[0, 1], [0, 2], [3, 5], [5, 4], [4, 3]], source = 0, destination = 5  ->  False
```

```
Start with:  def validPath(n, edges, source, destination):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100,000 vertices in a single long path (edges [i, i + 1]); the LeetCode limit is 200,000.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 19, Problem B

Class 19: Breadth-first search
Problem B (harder)

#994 Rotting Oranges

Medium. https://leetcode.com/problems/rotting-oranges/

A grid holds 0 (empty), 1 (fresh orange) and 2 (rotten orange). Every minute, each fresh orange that touches a rotten orange (up, down, left, right) becomes rotten. Return the minimum number of minutes until no fresh orange remains, or -1 if that is impossible.

```
Example: grid = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]  ->  4        grid = [[2, 1, 1], [0, 1, 1], [1, 0, 1]]  ->  -1        grid = [[0, 2]]  ->  0
Start with:  def orangesRotting(grid):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for A 10 x 10 grid (the LeetCode limit, n = 100 cells), a single rotten orange in a corner, all other cells fresh.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 20, Problem A

Class 20: Topological sort
Problem A (easier)

#207 Course Schedule

Medium. https://leetcode.com/problems/course-schedule/

There are numCourses courses numbered 0 to numCourses - 1. prerequisites[i] = [a, b] means you must take course b before course a. Return True if it is possible to finish all courses (that is, there is no cycle of prerequisites).

```
Example: numCourses = 2, prerequisites = [[1, 0]]  ->  True        numCourses = 2, prerequisites = [[1, 0], [0, 1]]  ->  False
```

```
Start with:  def canFinish(numCourses, prerequisites):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for V = 2,000 courses (the LeetCode limit) in one long chain of prerequisites, E = 1,999.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 20, Problem B

Class 20: Topological sort
Problem B (harder)

#210 Course Schedule II

Medium. https://leetcode.com/problems/course-schedule-ii/

The same setup as Course Schedule: prerequisites[i] = [a, b] means take course b before course a. Return one valid order in which you can take all the courses. If it is impossible (there is a cycle), return an empty list.

```
Example: numCourses = 4, prerequisites = [[1, 0], [2, 0], [3, 1], [3, 2]]  ->
[0, 1, 2, 3]   (or [0, 2, 1, 3])
```

```
Start with:  def findOrder(numCourses, prerequisites):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for V = 2,000 courses in a chain (E = 1,999).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 21, Problem A

Class 21: Shortest paths: Dijkstra
Problem A (easier)

#743 Network Delay Time

Medium. https://leetcode.com/problems/network-delay-time/

A network has n nodes labeled 1 to n. times[i] = [u, v, w] is a directed edge: a signal sent from u reaches v after w time units. A signal is sent from node k. Return the time until all n nodes have received it, or -1 if some node can never receive it.

```
Example: times = [[2, 1, 1], [2, 3, 1], [3, 4, 1]], n = 4, k = 2  ->  2
Start with:  def networkDelayTime(times, n, k):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for V = 100 nodes and E = 6,000 edges (the LeetCode limits), random edges with weights 1 to 100.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 21, Problem B

Class 21: Shortest paths: Dijkstra
Problem B (harder)

#1631 Path With Minimum Effort

Medium. https://leetcode.com/problems/path-with-minimum-effort/

heights is a grid of integers. You start at the top-left cell and want to reach the bottom-right cell, moving up, down, left or right. The effort of a route is the largest absolute height difference between two consecutive cells on it. Return the minimum effort over all routes.

```
Example: heights = [[1, 2, 2], [3, 8, 2], [5, 3, 5]]  ->  2
Start with:  def minimumEffortPath(heights):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for A 100 x 100 grid (the LeetCode limit, n = 10,000 cells) that is flat (every height 1) except the goal cell, so the search explores every cell at effort 0 before reaching the goal.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 22, Problem A

Class 22: Union-find
Problem A (easier)

#547 Number of Provinces

Medium. https://leetcode.com/problems/number-of-provinces/

There are n cities. isConnected is an n x n matrix: isConnected[i][j] = 1 if cities i and j are directly connected, else 0. A province is a group of cities connected directly or indirectly. Return the number of provinces.

```
Example: isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]  ->  2
isConnected = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]  ->  3
Start with:  def findCircleNum(isConnected):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 200 cities (the LeetCode limit), nothing connected except each city to itself (the worst case for scanning).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 22, Problem B

Class 22: Union-find
Problem B (harder)

#684 Redundant Connection

Medium. https://leetcode.com/problems/redundant-connection/

A tree with n nodes (labeled 1 to n) had one extra edge added, so the input is a list of n edges that contains exactly one cycle. Return the edge that can be removed so the rest is a tree. If several edges work, return the one that appears last in the input.

```
Example: edges = [[1, 2], [1, 3], [2, 3]]  ->  [2, 3]        edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]  ->  [1, 4]
Start with:  def findRedundantConnection(edges):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 1,000 edges (the LeetCode limit) forming a path 1-2-...-n with the cycle-closing edge [1, n] last.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 23: Tries
Problem A (easier)

#208 Implement Trie (Prefix Tree)

Medium. https://leetcode.com/problems/implement-trie-prefix-tree/

Implement a Trie: insert(word) stores a word; search(word) returns True if the exact word was inserted; startsWith(prefix) returns True if any inserted word begins with prefix. All strings use lowercase letters.

```
Example: insert("apple"); search("apple") -> True; search("app") -> False;
startsWith("app") -> True; insert("app"); search("app") -> True
```

```
Start with:  class Trie
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for 10,000 random lowercase words of 8 letters each (80,000 letters).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 23, Problem B

Class 23: Tries
Problem B (harder)

#212 Word Search II

Hard. https://leetcode.com/problems/word-search-ii/

Given an m x n board of letters and a list of words, return all the words that can be formed by a path of adjacent cells (horizontal or vertical). A cell may be used only once per word. Each word is distinct.

```
Example: board = [["o","a","a","n"], ["e","t","a","e"], ["i","h","k","r"],
["i","f","l","v"]], words = ["oath","pea","eat","rain"]  ->  ["eat", "oath"]
Start with:  def findWords(board, words):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for 10,000 random 8-letter words (as for Trie) stored in the trie before the search starts, on a 12 x 12 random board.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 24, Problem A

Class 24: Backtracking
Problem A (easier)

#78 Subsets

Medium. https://leetcode.com/problems/subsets/

Given a list nums of distinct integers, return all possible subsets (the power set), in any order. The result must not contain duplicate subsets.

```
Example: nums = [1, 2, 3]  ->  [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
Start with:  def subsets(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10 (the LeetCode limit): 1,024 subsets with 5,120 entries in total.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 24, Problem B

Class 24: Backtracking
Problem B (harder)

#51 N-Queens

Hard. https://leetcode.com/problems/n-queens/

Place n queens on an n x n chessboard so that no two queens attack each other (no two share a row, column or diagonal). Return every distinct solution, each as a list of n strings with "Q" for a queen and "." for an empty square.

```
Example: n = 4  ->  [[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]
```

```
Start with:  def solveNQueens(n):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 8: 92 solutions, each a list of 8 strings of 8 characters.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 25, Problem A

Class 25: Greedy algorithms
Problem A (easier)

#55 Jump Game

Medium. https://leetcode.com/problems/jump-game/

nums[i] is the maximum jump length from position i. Starting at index 0, return True if you can reach the last index, and False otherwise.

```
Example: nums = [2, 3, 1, 1, 4]  ->  True        nums = [3, 2, 1, 0, 4]  ->  False
```

```
Start with:  def canJump(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 values (the LeetCode limit), all 1 (reach moves one step at a time).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 25, Problem B

Class 25: Greedy algorithms
Problem B (harder)

#135 Candy

Hard. https://leetcode.com/problems/candy/

n children stand in a line, and ratings[i] is the rating of child i. Give each child at least one candy. A child with a higher rating than a neighbor must get more candies than that neighbor. Return the minimum total number of candies.

```
Example: ratings = [1, 0, 2]  ->  5   (candies 2, 1, 2)        ratings = [1, 2, 2]  ->  4   (candies 1, 2, 1)
Start with:  def candy(ratings):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 20,000 children (the LeetCode limit), ratings strictly increasing (the worst case for the size of the candy counts: 1, 2, ..., n).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 26, Problem A

Class 26: Intervals and sweep line
Problem A (easier)

#56 Merge Intervals

Medium. https://leetcode.com/problems/merge-intervals/

Given a list of intervals [start, end], merge all overlapping intervals and return the non-overlapping intervals that cover the same ranges. Intervals that touch at an endpoint (like [1, 4] and [4, 5]) overlap.

```
Example: intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]  ->  [[1, 6], [8, 10], [15, 18]]
Start with:  def merge(intervals):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 disjoint intervals (the LeetCode limit and the worst case: nothing merges).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 26, Problem B

Class 26: Intervals and sweep line
Problem B (harder)

#218 The Skyline Problem

Hard. https://leetcode.com/problems/the-skyline-problem/

Each building is [left, right, height]. Return the skyline: the list of key points [x, height] where the height of the outline changes, sorted by x. The last point has height 0. Consecutive key points must have different heights.

```
Example: buildings = [[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]  ->  [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]
Start with:  def getSkyline(buildings):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 10,000 buildings (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 27, Problem A

Class 27: Dynamic programming: 1D
Problem A (easier)

#198 House Robber

Medium. https://leetcode.com/problems/house-robber/

nums[i] is the money in house i along a street. You cannot rob two adjacent houses. Return the maximum amount you can rob.

```
Example: nums = [1, 2, 3, 1]  ->  4   (rob houses 0 and 2)        nums = [2, 7, 9, 3, 1]  ->  12   (rob houses 0, 2 and 4)
Start with:  def rob(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 100 houses (the LeetCode limit), values up to 400.

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 27, Problem B

Class 27: Dynamic programming: 1D
Problem B (harder)

#322 Coin Change

Medium. https://leetcode.com/problems/coin-change/

coins is a list of coin values (unlimited supply of each) and amount is a target. Return the fewest coins that make exactly amount, or -1 if it cannot be made.

```
Example: coins = [1, 2, 5], amount = 11  ->  3   (5 + 5 + 1)        coins = [2], amount = 3  ->  -1
```

```
Start with:  def coinChange(coins, amount):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for amount = 10,000 (the LeetCode limit) with coins [1, 2, 5].

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 28, Problem A

Class 28: Dynamic programming: sequences
Problem A (easier)

#300 Longest Increasing Subsequence

Medium. https://leetcode.com/problems/longest-increasing-subsequence/

Given a list nums, return the length of the longest strictly increasing subsequence (the numbers need not be next to each other, but must keep their order).

```
Example: nums = [10, 9, 2, 5, 3, 7, 101, 18]  ->  4   (for example 2, 3, 7, 101)
Start with:  def lengthOfLIS(nums):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 2,500 values (the LeetCode limit), strictly increasing (the worst case for dp values: 1, 2, ..., n).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 28, Problem B

Class 28: Dynamic programming: sequences
Problem B (harder)

#1143 Longest Common Subsequence

Medium. https://leetcode.com/problems/longest-common-subsequence/

Given two strings text1 and text2, return the length of their longest common subsequence: the longest sequence of characters that appears in both in the same order (not necessarily next to each other). Return 0 if there is none.

```
Example: text1 = "abcde", text2 = "ace"  ->  3   ("ace")        text1 = "abc",
text2 = "def"  ->  0
```

```
Start with:  def longestCommonSubsequence(text1, text2):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for m = n = 1,000 random lowercase strings (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 29, Problem A

Class 29: Dynamic programming: 2D tables
Problem A (easier)

#72 Edit Distance

Medium. https://leetcode.com/problems/edit-distance/

Given two words word1 and word2, return the minimum number of single-character operations (insert, delete or replace a character) needed to turn word1 into word2.

```
Example: word1 = "horse", word2 = "ros"  ->  3   (horse -> rorse -> rose -> ros)
Start with:  def minDistance(word1, word2):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for m = n = 500 random lowercase strings (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 29, Problem B

Class 29: Dynamic programming: 2D tables
Problem B (harder)

#10 Regular Expression Matching

Hard. https://leetcode.com/problems/regular-expression-matching/

Implement regular expression matching with "." (matches any single character) and "*" (matches zero or more copies of the character just before it). Return True if the whole string s matches the whole pattern p.

```
Example: s = "aa", p = "a"  ->  False        s = "aa", p = "a*"  ->  True
s = "ab", p = ".*"  ->  True
```

```
Start with:  def isMatch(s, p):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for m = n = 20 (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Name _____ Date _____ Class 30, Problem A

Class 30: Capstone: choosing the pattern
Problem A (easier)

#127 Word Ladder

Hard. https://leetcode.com/problems/word-ladder/

beginWord and endWord are words of the same length, and wordList is a list of allowed words. A move changes exactly one letter, and every word you reach must be in wordList. Return the number of words in the shortest sequence from beginWord to endWord (counting both ends), or 0 if no sequence exists.

```
Example: beginWord = "hit", endWord = "cog", wordList =
["hot","dot","dog","lot","log","cog"]  ->  5   (hit -> hot -> dot -> dog -> cog)
Start with:  def ladderLength(beginWord, endWord, wordList):
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for N = 3,125 words (all 5-letter words over the letters a to e), L = 5, beginWord "aaaaa" and endWord "eeeee".

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____

Class 30: Capstone: choosing the pattern
Problem B (harder)

#295 Find Median from Data Stream

Hard. https://leetcode.com/problems/find-median-from-data-stream/

Design a MedianFinder. addNum(num) adds an integer from a data stream; findMedian() returns the median of all numbers so far: the middle value if the count is odd, or the mean of the two middle values if it is even.

```
Example: addNum(1); addNum(2); findMedian() -> 1.5; addNum(3); findMedian() -> 2.0
```

```
Start with:  class MedianFinder
```

Python solution

Analysis

Time complexity

Auxiliary space

Memory estimate, for n = 50,000 numbers added (the LeetCode limit).

Input memory (bytes)

Auxiliary memory (bytes)

Total memory (bytes)

Failed attempts before asking for help: 1. _____ 2. _____